Mechanical and Aerospace Engineering
Machine Learning in Fluid Flow
Dr. Qiong Liu

Build and Analyze Your Own Convolutional Neural Network (CNN)

Luis C. Diaz Dominguez
800772032
luisper2@nmsu.edu
November 16th, 2025

Part 1: Modify the Provided Example

Network Architecture

For this homework, I modified the original CNN by adding an additional convolutional–pooling block, resulting in a total of three convolutional layers with increasing filter sizes (16, 32, and 64). I also changed the kernel size to 4×4 and replaced max-pooling with average-pooling in both models. The main architectural difference between Model 1 and Model 2 is the pooling size: Model 1 uses 2×2 pooling, while Model 2 uses 1×1 pooling. Finally, the activation functions were changed to `tanh` for all hidden layers and `sigmoid` for the output layer.

```
''' ____ Model 1 ____ '''
model_1 = Sequential()

''' Layers '''
# Layer 1
model_1.add(Convolution2D(16, kernel_size = (4, 4), strides = (1, 1),
    input_shape=(64, 64, 3), activation = 'tanh'))
model_1.add(AveragePooling2D(pool_size = (2, 2)))

# Layer 2
model_1.add(Convolution2D(32, kernel_size = (4, 4), strides = (1, 1),
    activation = 'tanh'))
model_1.add(AveragePooling2D(pool_size = (2, 2)))

# Layer 3
model_1.add(Convolution2D(64, kernel_size = (4, 4), strides = (1, 1),
    activation = 'tanh'))
model_1.add(AveragePooling2D(pool_size = (2, 2)))

# Flattening
model_1.add(Flatten())

# Dense
model_1.add(Dense(32, activation = 'tanh'))
model_1.add(Dense(neurons, activation = 'sigmoid'))

''' ____ Model 2 ____ (Reducing just Pooling size) '''
model_2 = Sequential()

''' Layers '''
# Layer 1
model_2.add(Convolution2D(16, kernel_size = (4, 4), strides = (1, 1),
    input_shape=(64, 64, 3), activation = 'tanh'))
model_2.add(AveragePooling2D(pool_size = (1, 1)))

# Layer 2
model_2.add(Convolution2D(32, kernel_size = (4, 4), strides = (1, 1),
    activation = 'tanh'))
model_2.add(AveragePooling2D(pool_size = (1, 1)))

# Layer 3
model_2.add(Convolution2D(64, kernel_size = (4, 4), strides = (1, 1),
    activation = 'tanh'))
model_2.add(AveragePooling2D(pool_size = (1, 1)))

# Flattening
model_2.add(Flatten())

# Dense
model_2.add(Dense(32, activation = 'tanh'))
model_2.add(Dense(neurons, activation = 'sigmoid'))
```

Training Parameters

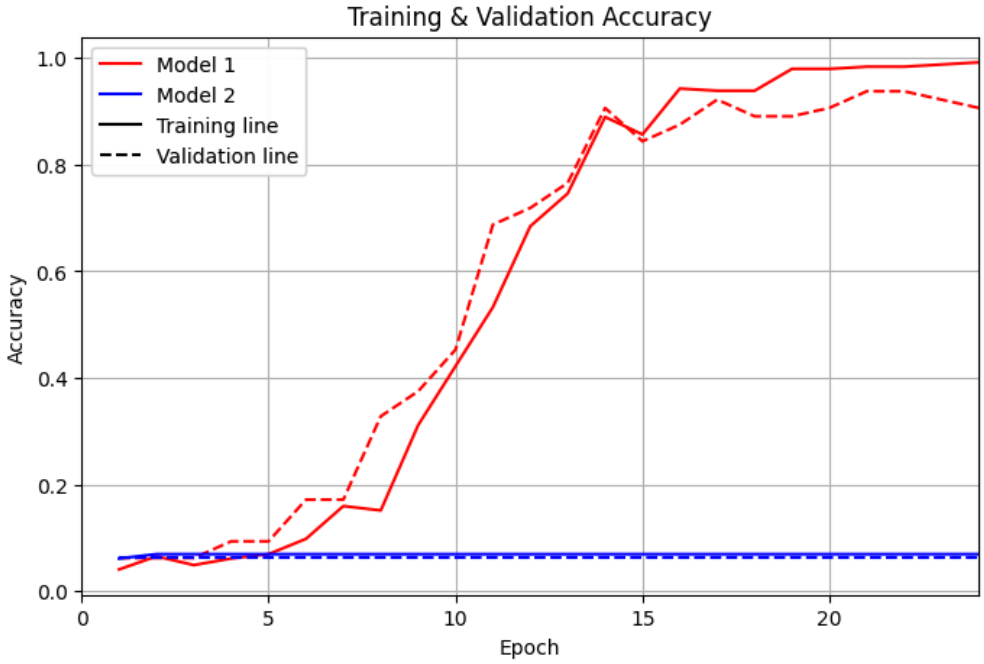
For training, I used the AdamW optimizer, an improved version of Adam that incorporates decoupled weight decay. Each model was trained for 24 epochs, using 20 validation steps to better estimate validation performance. These changes were introduced to accelerate convergence and improve generalization.

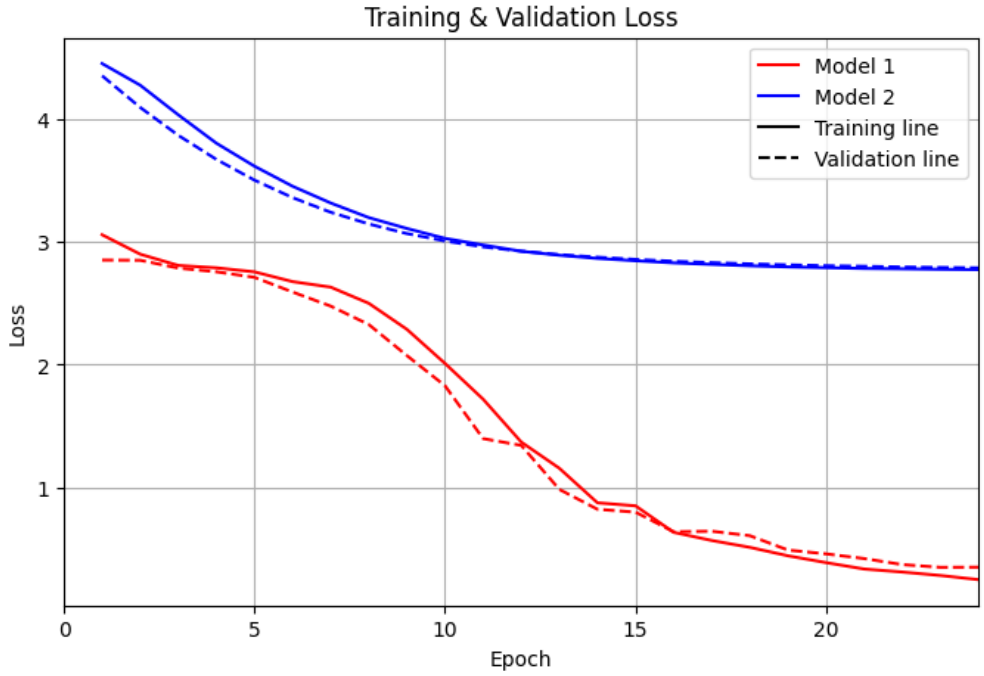
```
import time

model_1.compile(loss = 'categorical_crossentropy', optimizer =
    'adamW', metrics = ['accuracy'])
model_2.compile(loss = 'categorical_crossentropy', optimizer =
    'adamW', metrics = ['accuracy'])

histories = []
models = [model_1, model_2]

for idx, mdl in enumerate(models, start = 1):
    start = time.time()

    history = mdl.fit(
        trainSet,
        steps_per_epoch = len(trainSet),
        epochs = 24,
        validation_data = testSet,
        validation_steps = 20,
        verbose = 0)
    histories.append(history)

    end = time.time()

    print(f'Total Time: (model {idx}) {round(end-start)}s')

model_1, model_2 = models
```

Visualization and Analysis

Training Curves

The plots below show the training and validation accuracy and loss for both models. Model 1 demonstrates strong and consistent learning, whereas Model 2 fails to improve beyond chance level.



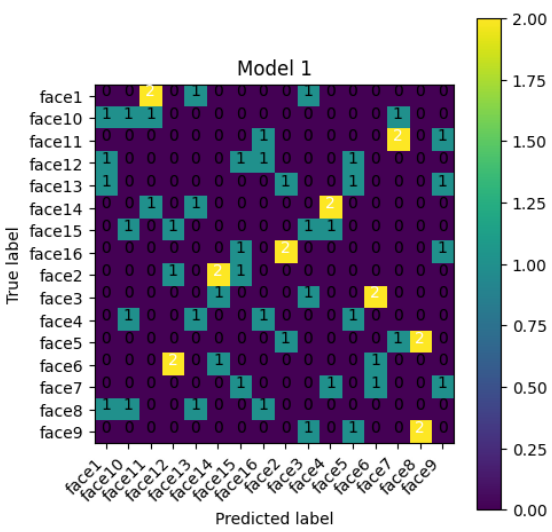


Performance and Metrics

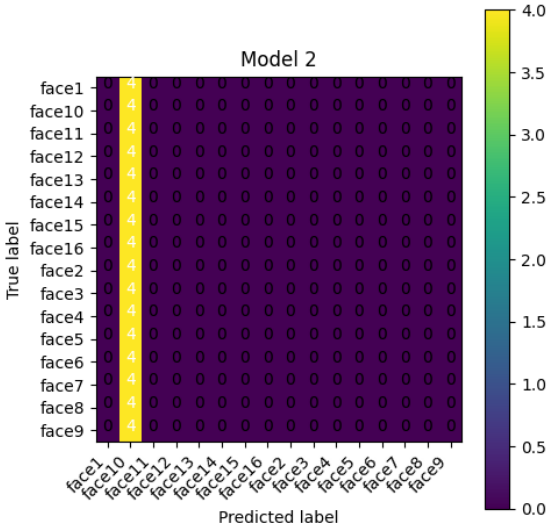
Model	Train Accuracy	Validation Accuracy	Test Accuracy (Loss)
Model 1	0.9918	0.9062	0.9062 (0.3512)
Model 2	0.0697	0.0625	0.0625 (2.7875)

Table 1: Final performance comparison of Model 1 and Model 2.

Confusion Matrices



(a) Model 1 Confusion Matrix

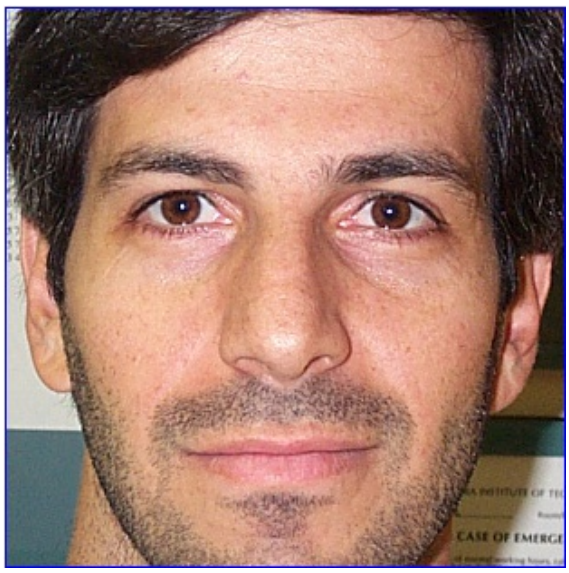


(b) Model 2 Confusion Matrix

Visual Results

Below are examples of correctly and incorrectly classified images. Model 1 misclassified only one face in the first experiment and later achieved perfect classification on a second training run. Model 2 consistently struggled, correctly identifying only one individual (face 10).

(a) Incorrect Example (Model 1)



(b) Only Correct Example (Model 2)

Reflection and Report

During this project, I modified the CNN architecture and training setup from the original lecture example to better understand how different design choices influence model behavior. I introduced a deeper architecture with three convolutional layers, changed the kernel size to 4×4, and replaced max-pooling with average-pooling. I also compared two versions of the network by varying the pooling size: Model 1 used 2×2 pooling, while Model 2 used 1×1. In addition, I changed the activation functions to `tanh` and `sigmoid` to explore their impact on nonlinearity and learning stability. For optimization, I used the AdamW optimizer and trained each model for 24 epochs.

These modifications significantly affected performance. Model 1 showed strong convergence, achieving over 90% validation and test accuracy. Its confusion matrix indicates stable feature extraction across multiple faces. In contrast, Model 2 failed almost completely, never improving beyond random guessing. The lack of spatial downsampling (due to 1×1 pooling) resulted in excessive feature-map dimensionality, making learning inefficient and unstable. This side-by-side comparison highlighted how critical pooling layers are for reducing spatial complexity and improving generalization.

Through this exercise, I gained a deeper intuition about CNN architecture design. I learned how convolutional depth, pooling size, and activation functions influence the model's ability to extract discriminative features. I also observed how training curves, confusion matrices, and misclassification examples collectively reveal model strengths and weaknesses. Overall, this assignment strengthened my understanding of how CNNs process image data and how architectural decisions directly impact performance.